

Reading Exercise 01 — The First Walk

You are not allowed to understand anything in this exercise. You are only allowed to look.

This is a timed exercise. Each step has a time limit. When the time is up, stop, whether or not you feel done. The discipline of stopping is part of the exercise.

What this exercise trains

The instinct most students bring to an unfamiliar codebase is: *read until you understand*. That instinct fails at scale. It produces anxiety, not orientation.

This exercise trains a different instinct: *look before you read. Map before you navigate. Know the shape before you know the content*. It is the difference between a tourist who opens the map before leaving the hotel and one who walks until lost.

Step 1 — The directory [5 minutes]

Open `practice/tcc/`. Look at the files. Do not open any of them.

```
ls -lh practice/tcc/*.c practice/tcc/*.h | sort -k5 -rh
```

This lists all source files sorted by size, largest first.

Write down, without opening any file:

- How many `.c` files are there? How many `.h` files?
- What is the largest `.c` file? The smallest?
- Group the files by what you guess they do, based only on their names. Aim for four or five groups.

Time is up. Stop.

Step 2 — Sixty seconds each [12 minutes total]

For each file in this list, open it, look at it for exactly 60 seconds, then close it. No more. Do not read; look. Scan for structure, for patterns, for what's repeated, for what's surprising.

```
tcc.c      tccgen.c    tccpp.c
tccelf.c   tccrun.c    libtcc.c
tcc.h      x86_64-gen.c i386-gen.c
arm-gen.c  tccasm.c    tcctools.c
```

After each file, write one sentence — just one — about what you saw. Not what you understood. What you saw.

Step 3 — Revise your map [5 minutes]

Return to your groups from Step 1. Revise them based on what you saw. Add, remove, rename.

Write down: what changed? Which file surprised you most, and why?

Step 4 — Size intuition [5 minutes]

Without looking at line counts, guess: which three files have the most lines of code? Write your guesses.

Then check:

```
wc -l practice/tcc/*.c | sort -rn | head -10
```

Write down: how close were you? What does the actual distribution tell you about where TCC's complexity lives?

Step 5 — The question you have [2 minutes]

After this walk, you have seen the whole directory but understood almost none of it.

Write down: the single question you most want answered about how TCC works. Not "how does everything work" — the one specific thing that made you curious.

That question is your entry point for Reading Exercise 02.

Debrief

This exercise is complete when you have written your answers to each step. The writing is not decoration — it forces the looking to become more precise.

A directory walk like this, done carefully, gives you more orientation in 30 minutes than three hours of undirected reading. It tells you where the mass is, what the structure is, and where to go next.

You now know the shape of TCC. You don't yet know what it does. That comes next.

Next: RE-02 — Finding main() — following the program's beginning.